

14.2 ReAct and Tool Use

What ReAct is, why a chain of thought is not enough, the tool loop, then `calc[17*24]=408`.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

ReAct (Yao et al.: *Reason + Act*) is the control pattern after retrieve-then-stuff. Note **14.1** taught RAG as one lookup, then generate. Here the model can **call a tool**, read what comes back, and only then continue writing.

1. What ReAct is

A **chain of thought (CoT)** is extra tokens the model writes before the answer (Week 13). Those tokens are still guesses. If the thought says " $17 \times 24 = 428$ ", the algebra can be wrong and the prose still looks careful.

ReAct interleaves three kinds of line:

- **Thought:** scratch work in language. Not a tool. Not the answer.
- **Action:** a **structured** call the environment can parse, for example `calc[17*24]`, `search["library hours"]`, `lookup[d3]`.
- **Observation:** a string the **environment** returns. The model did not write it.

The model is not only talking to itself. It is talking to a tool, then reading. Contrast **ReAct vs CoT vs self-consistency**. CoT can invent a fact. Self-consistency votes k invented traces. ReAct is supposed to **wait for the observation**.

RAG retrieve is **one** tool. A calculator is another. A code runner is another. The control pattern is the same.

2. Why we use it

Language models are bad at exact arithmetic, at looking up a fresh PDF, and at anything that needs a **world** (time, a database, a compiler). You can try to bake those skills into weights. That is slow and still drifts.

Tools fix a different slice of the problem than RAG stuffing:

- **RAG (14.1):** one retrieve, then one generate. The passages are already in the prompt before the first answer token.
- **ReAct:** the model **decides** when to call retrieve, or calc, or both, and it can call again after it reads.

Use ReAct when the next step depends on an observation you do not have yet. Use stuffed RAG when you already know you want top- k passages in the prompt.

3. Architecture

There are three pieces.

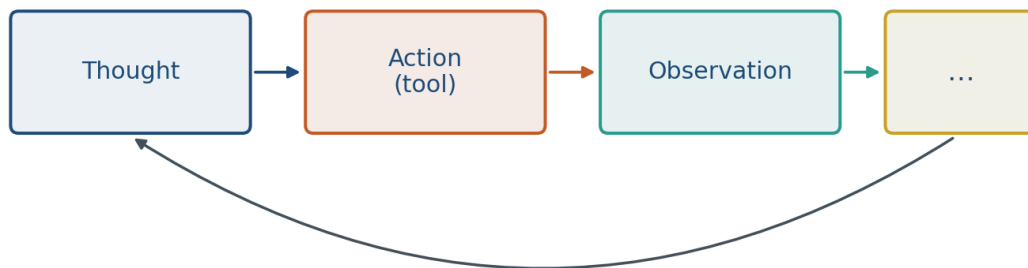
1. **Policy (the LLM)**. Samples the next line given the transcript so far.
2. **Parser**. Turns an Action line into a tool name and arguments. If the parse fails, the observation is an error string, not a silent skip.
3. **Tools**. Each tool is a function: arguments in, string out. Lab 14's `calc` only matches `[0-9+\s*/]`. That restriction is the point.

The transcript is the memory:

$$\text{context}_{t+1} = \text{context}_t + \text{Thought}_t + \text{Action}_t + \text{Observation}_t.$$

Stop when the model emits a final **Answer**, or when you hit a step cap.

ReAct: think, call a tool, read the result, repeat



Karpathy's calculator / interpreter demos are this loop in a product: special tokens, a tool runs, text comes back, the model continues.

4. How it works, step by step

Question: what is 17×24 ? Do **not** multiply in the thought.

1. **Thought**: I need a product; use the calculator.
2. **Action**: `calc[17*24]`
3. **Environment**: run the dummy calculator. **Observation**: `408`
4. **Thought**: Use 408; do not recompute.

5. Answer: 408

If step 4 says “that is 428” while the observation is 408, the loop is **unfaithful**: the observation was ignored (note **13.3**).

Lab 14’s library minutes: weekday close 23:00 from retrieval, then `calc[23*60]`, observation 1380. The answer must contain 1380 **from the observation**, plus a citation `[d1]` from RAG. Two tools, one loop.

5. Mathematical formulas

Let c_t be the transcript after t rounds. The model samples a thought and an action,

$$(\tau_t, a_t) \sim p_\theta(\cdot \mid c_t).$$

The environment is deterministic in this class:

$$o_t = \text{Tool}(a_t).$$

Then $c_{t+1} = c_t \oplus \tau_t \oplus a_t \oplus o_t$. A final answer y is sampled from $p_\theta(y \mid c_T)$.

There is **no** extra trained module today. ReAct is a **decoding protocol** plus tools. (You *can* SFT on ReAct traces later; Lab 14 does not.)

Compare CoT: $y \sim p_\theta(\text{steps, answer} \mid q)$ with no o_t . Compare stuffed RAG: one retrieve $\hat{z}_{1:k}$, then $y \sim p_\theta(y \mid q, \hat{z}_{1:k})$ with no loop.

6. Positive points and negative points

Positive.

- Exact operations (arithmetic, code, SQL) can be delegated instead of guessed.
- The next thought can depend on a real observation.
- You can mix retrieve, calc, and lookup without stuffing everything first.
- The transcript is an audit log: Thought / Action / Observation / Answer.

Negative.

- New bugs: malformed actions, wrong tool, ignored observation, infinite loops.
- Each tool call adds latency.
- A dummy `eval` on free Python is a security hole. Lab 14’s regex is the safe cartoon.
- If the thought already contains the product, `calc` is decoration and you did not test the tool.
- Mixing **Tree of Thoughts** (search over thoughts) with ReAct (call the world) under one name hides the knobs.

When not to use it. A single retrieve-then-answer with no later decision: stuffed RAG is simpler. A pure style rewrite: SFT, not a calculator.

7. Why tools change the error modes

CoT can invent a product. A calc tool can return the product **if** the model writes the right call. Parse actions strictly (regex or JSON). Cap the number of steps. Log every observation. If the parse fails, return `ERR` and let the model retry once; do not silently drop the call.

Write the five-line log on the board for every demo: Thought, Action, Observation, Thought, Answer. If a line is missing, you did not run ReAct; you ran CoT with a tool-shaped string.

ToT searches among thoughts. ReAct calls the world. You can combine them, but name the knobs separately. For the project, one tool plus a checker is enough.

8. Teaching this note

About **45 minutes** at the board, then **~8 minutes** of video. Lab 14 can follow in the same two-hour block. Students have not seen ReAct as a method before. Do **14.1** first.

- **0–10 min.** What / why: CoT can invent; a tool returns a string the model did not write.
 - **10–22 min.** Architecture: policy, parser, tools. Write the context-update formula. Five-line log.
 - **22–34 min.** Worked 17×24 with `calc`, then Lab 14's `23*60=1380`. Unfaithful next-thought.
 - **34–45 min.** Pros / cons. ReAct vs stuffed RAG vs ToT. Step cap and `ERR`.
 - **Then** play Karpathy intro **27:43–35:00** (calculator, interpreter, browser). Pause when a special token launches a tool and an observation returns.
-

9. Worked example

Question: what is 17×24 ?

role	text
Thought	I need a product; use the calculator.
Action	<code>calc[17*24]</code>
Observation	<code>408</code>
Thought	Use 408; do not recompute.
Answer	408

A second board log: `60/12` shuttle loops. Action `calc[60/12]`, observation `5.0`, cite `[d2]`.

If the second thought says 428 while the observation is 408, final-answer grading might still pass if it later copies 408, or fail on 428. Either way the **trace** is the bug.

10. Where students get stuck

- Multiplying in the Thought and using `calc` as decoration.
 - Swallowing a parse error (`ERR`) and answering anyway.
 - Calling stuffed RAG “ReAct” because retrieval happened. ReAct is the **loop**.
 - Mixing ToT with ReAct under one name.
-

11. Video

Watch [Andrej Karpathy, Intro to Large Language Models](#), 27:43–35:00 (tool use: browser, calculator, interpreter).

Pause on the calculator and on the Python interpreter: emit a call, get an observation, continue. Same URL as **14.1**; that note is the cosine index, this note is the control loop.

12. Practice

1. In one sentence each: what ReAct is, and how it differs from chain-of-thought.
2. Why must the observation be appended to the context before the next thought is sampled?
3. Write the five-line log for 12×11 using `calc`. What is the observation?
4. The observation is 408 and the next thought says 428. Name the bug. Is it a tool bug or a reader bug?
5. When would stuffed RAG (note 14.1) be the better design than a ReAct retrieve-tool?
6. Lab 14’s `calc` rejects letters. Why is that a feature for this class?